Метод опорных векторов

Данила Кочнев

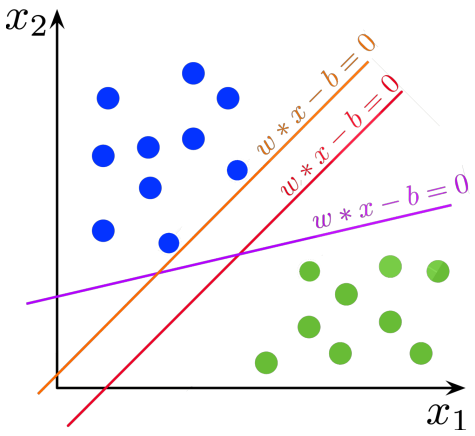
7.11.2026

Воспоминания о линейных классификаторах

1. Дана выборка $\{(\vec{\mathbf{x}}_i, y_i)\}$, причем пусть метки $y_i \in \{-1, 1\}$
2. Определим прогноз линейной модели как $\hat{y}(\vec{\mathbf{x}}_i) = \operatorname{sgn}(\vec{\mathbf{w}}^\top \vec{\mathbf{x}} + b)$
3. Тогда условие верной классификации имеет вид

$$\operatorname{sgn} \hat{y}_i = \operatorname{sgn} y_i \iff \underbrace{y_i(\vec{\mathbf{w}}^\top \vec{\mathbf{x}}_i + b)}_{\text{Margin}_i} > 0$$

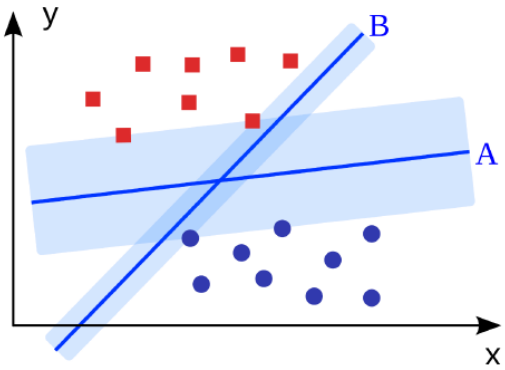
4. Если выборка *линейно разделима*, то которую из разделяющих гиперплоскостей выбрать?



[source]

Геометрическая идея SVC

1. Расстояние от $\vec{\mathbf{x}}_i$ до гиперплоскости $\vec{\mathbf{w}}^\top \vec{\mathbf{x}} + b = 0$ равно $d_i = \frac{|\vec{\mathbf{w}}^\top \vec{\mathbf{x}} + b|}{\|\vec{\mathbf{w}}\|}$
2. Тогда можно отыскать *наиболее уверенную* разделяющую гиперплоскость:
$$\min_i d_i = \min_i \frac{|\vec{\mathbf{w}}^\top \vec{\mathbf{x}}_i + b|}{\|\vec{\mathbf{w}}\|} \to \max_{\vec{\mathbf{w}}, b}$$
3. Если заранее договориться о том, что $\min_i |\vec{\mathbf{w}}^\top \vec{\mathbf{x}}_i + b| = 1$, то задача сводится к задаче $\frac{1}{\|\vec{\mathbf{w}}\|} \to \max$, или $\|\vec{\mathbf{w}}\|^2 \to \min$



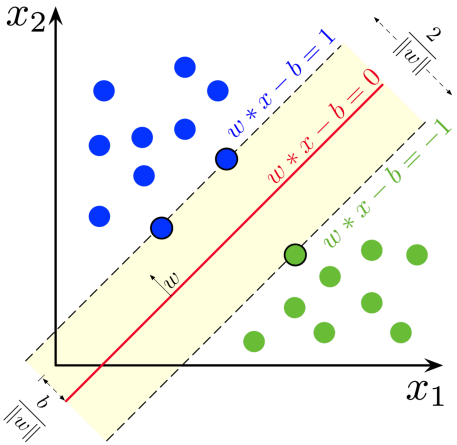
[source]

Hard-margin SVC

1. Можно максимизировать ширину полосы при условии верной классификации всех объектов:
$$\forall\, i \;\; y_i(\vec{\mathbf{w}}^\top \vec{\mathbf{x}}_i + b) \geq 1$$

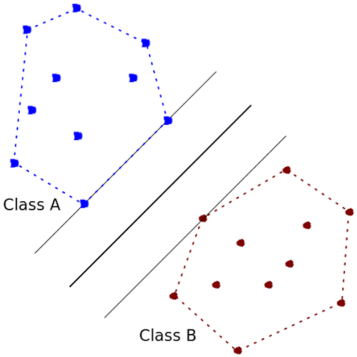
2. Тогда итоговая постановка оптимизационной задачи на линейно разделимой выборке:
$$\begin{cases} \frac{1}{2}\|\vec{\mathbf{w}}\|^2 \to \min\limits_{\vec{\mathbf{w}},b} \\ \forall\, i \;\; y_i(\vec{\mathbf{w}}^\top \vec{\mathbf{x}}_i + b) \geq 1 \end{cases}$$

3. Наблюдения, для которых $y_i(\vec{\mathbf{w}}^\top \vec{\mathbf{x}}_i + b) = 1$, называются *опорными векторами*

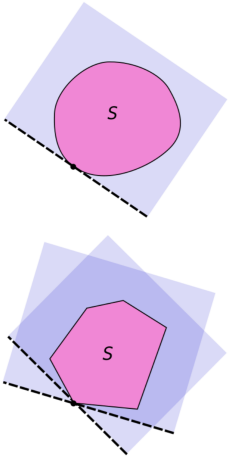


[source]

Про смысл названия и связь с выпуклым анализом



[source]



[source]

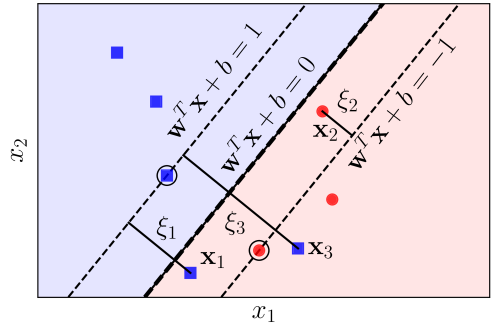
Soft-margin SVC – постановка

1. В случае линейно неразделимой выборки можно согласиться на неверную классификацию:

$$\forall\, i \;\; y_i(\vec{\mathbf{w}}^\top \vec{\mathbf{x}}_i + b) \geq 1 - \xi_i, \;\; \xi_i \geq 0$$

2. И наложить штраф за большие ошибки:

$$\begin{cases} \frac{1}{2}\|\vec{\mathbf{w}}\|^2 + C\sum\limits_{i=1}^{n} \xi_i \to \min\limits_{\vec{\mathbf{w}}, b, \xi_1, \ldots, \xi_n} \\ \forall\, i \;\; y_i(\vec{\mathbf{w}}^\top \vec{\mathbf{x}}_i + b) \geq 1 - \xi_i \\ \forall\, i \;\; \xi_i \geq 0 \end{cases}$$



[source]

Soft-margin SVC – решение – I – Hinge Loss

1. Перепишем ограничения в виде

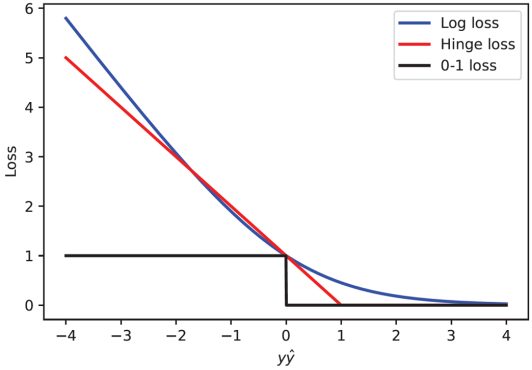
$$\begin{cases} \forall\, i \quad \xi_i \geq 1 - y_i(\vec{\mathbf{w}}^\top \vec{\mathbf{x}}_i + b) \\ \forall\, i \quad \xi_i \geq 0 \end{cases}$$

2. Тогда оптимальный выбор штрафов:

$$\xi_i^* = \max\left\{0,\ 1 - y_i(\vec{\mathbf{w}}^\top \vec{\mathbf{x}}_i + b)\right\}$$

3. А задачу soft-margin SVC можно переписать в безусловном виде

$$\frac{1}{2}\|\vec{\mathbf{w}}\|^2 + C\sum_{i=1}^{n} \max\left\{0,\ 1 - y_i(\vec{\mathbf{w}}^\top \vec{\mathbf{x}}_i + b)\right\} \to \min_{\vec{\mathbf{w}}, b}$$



[source]

Soft-margin SVC – решение – II – KKT

1. Возвращаясь к исходной задаче с ограничениями-неравенствами, можно составить лагранжиан:

$$\mathcal{L} = \frac{1}{2}\|\vec{\mathbf{w}}\|^2 + C\sum_{i=1}^{n}\xi_i + \sum_{i=1}^{n}\lambda_i\left(1 - \xi_i - y_i(\vec{\mathbf{w}}^\top\vec{\mathbf{x}}_i + b)\right) - \sum_{i=1}^{n}\mu_i\xi_i$$

2. Необходимое условие экстремума – условия KKT:

$$\nabla_{\vec{\mathbf{w}}}\mathcal{L} = 0 \iff \vec{\mathbf{w}} - \sum_{i=1}^{n}\lambda_i y_i\vec{\mathbf{x}}_i = 0 \iff \vec{\mathbf{w}} = \sum_{i=1}^{n}\lambda_i y_i\vec{\mathbf{x}}_i$$

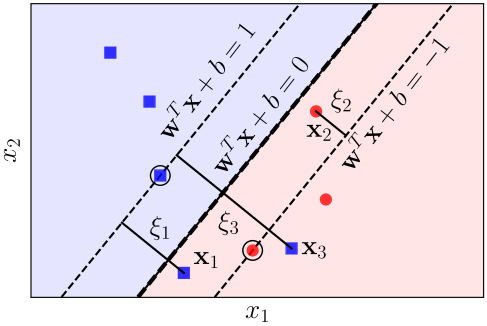
$$\frac{\partial\mathcal{L}}{\partial b} = 0 \iff -\sum_{i=1}^{n}\lambda_i y_i = 0 \iff \sum_{i=1}^{n}\lambda_i y_i = 0$$

$$\frac{\partial\mathcal{L}}{\partial\xi_i} = 0 \iff C - \lambda_i - \mu_i = 0 \iff \lambda_i + \mu_i = C$$

SVC – типизация объектов

Поскольку $\lambda_i \geq 0$, $\mu_i \geq 0$, $C \geq 0$ и из последнего KKT-условия имеем следующую типизацию объектов:

1. $\lambda_i = 0 \implies \mu_i = C \implies \xi_i = 0 \implies M_i \geq 1$ (периферийные / неинформативные объекты)
2. $0 < \lambda_i, \mu_i < C \implies \xi_i = 0$ и $M_i = 1$ (опорные граничные объекты)
3. $\lambda_i = C \implies \mu_i = 0 \implies \xi_i > 0 \implies M_i < 1$ (опорные объекты-нарушители)



[source]

Soft margin SVC – двойственная задача

Подставляя в лагранжиан KKT-условия, получаем двойственную задачу:

$$\mathcal{L} = \frac{1}{2}\|\vec{\mathbf{w}}\|^2 + C\sum_{i=1}^{n}\xi_i + \sum_{i=1}^{n}\lambda_i\Big(1 - \xi_i - y_i(\vec{\mathbf{w}}^\top\vec{\mathbf{x}}_i + b)\Big) - \sum_{i=1}^{n}\mu_i\xi_i =$$

$$= \frac{1}{2}\vec{\mathbf{w}}^\top\vec{\mathbf{w}} + \sum_{i=1}^{n}(C\xi_i - \lambda_i\xi_i - \mu_i\xi_i) + \sum_{i=1}^{n}\lambda_i - \sum_{i=1}^{n}\Big(\lambda_i y_i(\vec{\mathbf{w}}^\top\vec{\mathbf{x}}_i + b)\Big) =$$

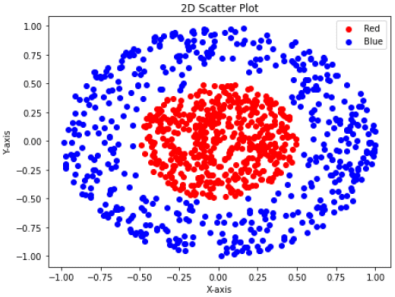
$$= \frac{1}{2}\vec{\mathbf{w}}^\top\vec{\mathbf{w}} + \sum_{i=1}^{n}\lambda_i - \vec{\mathbf{w}}^\top\sum_{i=1}^{n}\lambda_i y_i\vec{\mathbf{x}}_i + b\sum_{i=1}^{n}\lambda_i y_i =$$

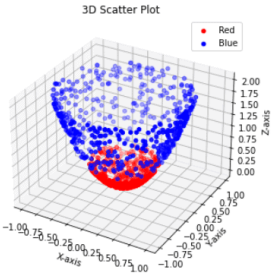
$$= \frac{1}{2}\vec{\mathbf{w}}^\top\vec{\mathbf{w}} + \sum_{i=1}^{n}\lambda_i - \vec{\mathbf{w}}^\top\vec{\mathbf{w}} = -\frac{1}{2}\vec{\mathbf{w}}^\top\vec{\mathbf{w}} + \sum_{i=1}^{n}\lambda_i =$$

$$= -\frac{1}{2}\sum_{i,j}\lambda_i\lambda_j y_i y_j\vec{\mathbf{x}}_i^\top\vec{\mathbf{x}}_j + \sum_{i=1}^{n}\lambda_i \longrightarrow \max_{\substack{0\le\lambda_1,\ldots,\lambda_n\le C \\ \sum\lambda_i y_i = 0}}$$

Kernel trick – неявное повышение$^?$ размерности

1. В некоторых ситуациях бывает удачно добавить в данные $(\vec{\mathbf{x}}_i, y_i)$ дополнительных нелинейных признаков $\varphi(\vec{\mathbf{x}}_i)$
2. Иногда в новом пространстве $H = \varphi(X)$ выборку получается лучше разделить





[source]

Ядерный SVC – I

1. Если мы используем вместо старых признаков $\vec{\mathbf{x}}$ новые $\varphi(\vec{\mathbf{x}})$, двойственная SVC-задача преобразуется к виду

$$-\frac{1}{2}\sum_{i,j}\lambda_i\lambda_j y_i y_j \underbrace{\varphi(\vec{\mathbf{x}}_i)^\top\varphi(\vec{\mathbf{x}}_j)}_{K_{ij}} + \sum_{i=1}^{n}\lambda_i \longrightarrow \max_{\substack{0\le\lambda_1,\ldots,\lambda_n\le C\\ \sum\lambda_i y_i=0}}$$

2. Тогда нам в реальности достаточно знать только матрицу Грама новых признаков $K_{ij} = \varphi(\vec{\mathbf{x}}_i)^\top\varphi(\vec{\mathbf{x}}_j)$, а не учить модель в *спрямляющем пространстве* H с нуля
3. Более того, вместо *спрямляющего отображения* φ можно сразу подбирать функцию $K(\vec{\mathbf{x}}_1, \vec{\mathbf{x}}_2)$, которая и генерирует эту матрицу

Ядерный SVC – II

1. Если мы используем вместо старых признаков $\vec{\mathbf{x}}$ новые $\varphi(\vec{\mathbf{x}})$, двойственная SVC-задача преобразуется к виду

$$-\frac{1}{2}\sum_{i,j}\lambda_i\lambda_j y_i y_j \underbrace{\varphi(\vec{\mathbf{x}}_i)^\top\varphi(\vec{\mathbf{x}}_j)}_{K_{ij}} + \sum_{i=1}^{n}\lambda_i \longrightarrow \max_{\substack{0\leq\lambda_1,\ldots,\lambda_n\leq C\\ \sum\lambda_i y_i=0}}$$

2. Из необходимого условия экстремума $\vec{\mathbf{w}}^* = \sum\lambda_i^* y_i\varphi(\vec{\mathbf{x}}_i)$, а b^* можно посчитать по любому опорному граничному объекту (у которого $0 < \lambda_i^* < C$) как $b^* = y_i - (\vec{\mathbf{w}}^*)^\top\vec{\mathbf{x}}_i$
3. А тогда прогноз имеет вид $\hat{y}(\vec{\mathbf{x}}) = \operatorname{sgn}\big(b^* + \sum\lambda_i^* y_i \underbrace{\varphi(\vec{\mathbf{x}}_i)^\top\varphi(\vec{\mathbf{x}})}_{K(\vec{\mathbf{x}}_i,\vec{\mathbf{x}})}\big)$

Ядра – чистый изумруд

1. Функцию $K : X \times X \to \mathbb{R}$ будем называть *ядром*, если
   - $K \in C(X \times X)$
   - $\forall \vec{\mathbf{x}}_1, \vec{\mathbf{x}}_2 \in X \quad K(\vec{\mathbf{x}}_1, \vec{\mathbf{x}}_2) = K(\vec{\mathbf{x}}_2, \vec{\mathbf{x}}_1)$
   - $\forall \vec{\mathbf{x}}_1, \ldots, \vec{\mathbf{x}}_n \in X \;\; \forall c_1, \ldots, c_n \in \mathbb{R} \quad \sum_{i=1}^{n} \sum_{j=1}^{n} c_i c_j K(\vec{\mathbf{x}}_i, \vec{\mathbf{x}}_j) \geq 0$
2. Комбинации ядер с неотрицательными коэффициентами, произведения ядер и поточечные пределы ядер – тоже ядра

Две неформальные теоремы о ядрах из функционального анализа

Mercer's theorem. Если *что-то с чем-то*, то использование спрямляющего отображения φ эквивалентно использованию некоторого ядра K.

Representer theorem. Аргминимум любой *разумно устроенной* функции потерь выражается в виде функции от линейной комбинации ядер, вычисленных в точках обучающей выборки.

Примеры ядер – I

1. **Линейное ядро:** $K(\vec{\mathbf{x}}_1, \vec{\mathbf{x}}_2) = \vec{\mathbf{x}}_1^\top \vec{\mathbf{x}}_2$ (порождает стандартный линейный SVC)

Примеры ядер – I

1. **Линейное ядро:** $K(\vec{\mathbf{x}}_1, \vec{\mathbf{x}}_2) = \vec{\mathbf{x}}_1^\top \vec{\mathbf{x}}_2$ (порождает стандартный линейный SVC)

2. **Полиномиальное ядро степени d:** $K(\vec{\mathbf{x}}_1, \vec{\mathbf{x}}_2) = \left(1 + \vec{\mathbf{x}}_1^\top \vec{\mathbf{x}}_2\right)^d$
   Можно показать, что тогда соответствующее спрямляющее отображение состоит из всевозможных (взвешенных) одночленов степени не выше d, то есть для примера при $d = 2$ и $\vec{\mathbf{x}} \in \mathbb{R}^2$ будем иметь $H = \mathbb{R}^6$, причем:
   $$K(\vec{\mathbf{x}}_1, \vec{\mathbf{x}}_2) = \left(1 + \vec{\mathbf{x}}_1^\top \vec{\mathbf{x}}_2\right)^2 \iff \varphi(\vec{\mathbf{x}})^\top = \begin{bmatrix} 1 & \sqrt{2}x_1 & \sqrt{2}x_2 & \sqrt{2}x_1x_2 & x_1^2 & x_2^2 \end{bmatrix}$$

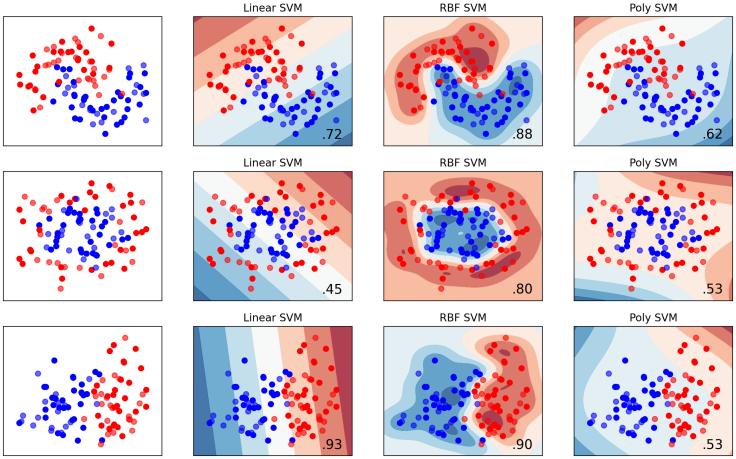
Примеры ядер – I

1. **Линейное ядро:** $K(\vec{x}_1, \vec{x}_2) = \vec{x}_1^\top \vec{x}_2$ (порождает стандартный линейный SVC)

2. **Полиномиальное ядро степени d:** $K(\vec{x}_1, \vec{x}_2) = \left(1 + \vec{x}_1^\top \vec{x}_2\right)^d$
   Можно показать, что тогда соответствующее спрямляющее отображение состоит из всевозможных (взвешенных) одночленов степени не выше d, то есть для примера при $d = 2$ и $\vec{x} \in \mathbb{R}^2$ будем иметь $H = \mathbb{R}^6$, причем:
   $$K(\vec{x}_1, \vec{x}_2) = \left(1 + \vec{x}_1^\top \vec{x}_2\right)^2 \iff \varphi(\vec{x})^\top = \begin{bmatrix} 1 & \sqrt{2}x_1 & \sqrt{2}x_2 & \sqrt{2}x_1x_2 & x_1^2 & x_2^2 \end{bmatrix}$$

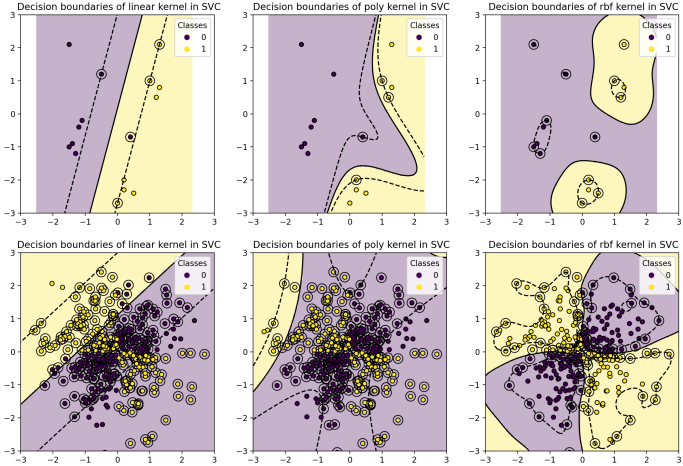
3. **RBF-ядро, или гауссово ядро:** $K(\vec{x}_1, \vec{x}_2) = e^{-\gamma\|\vec{x}_1 - \vec{x}_2\|^2}$
   Можно показать, что тогда соответствующее спрямляющее отображение состоит из всевозможных (взвешенных) одночленов, умноженных на $e^{-\gamma\|\vec{x}\|^2}$, то есть для примера при $\gamma = 1$ и $\vec{x} \in \mathbb{R}^1$ будем иметь $H = \mathbb{R}^\infty$, причем:
   $$K(x_1, x_2) = e^{-|x_1 - x_2|^2} \iff \varphi(x)^\top = e^{-x^2} \begin{bmatrix} 1 & x & \frac{1}{\sqrt{2}}x^2 & \frac{1}{\sqrt{6}}x^3 & \frac{1}{\sqrt{24}}x^4 & \dots \end{bmatrix}$$

Примеры ядер – II



[source]

Примеры ядер – III



[source]

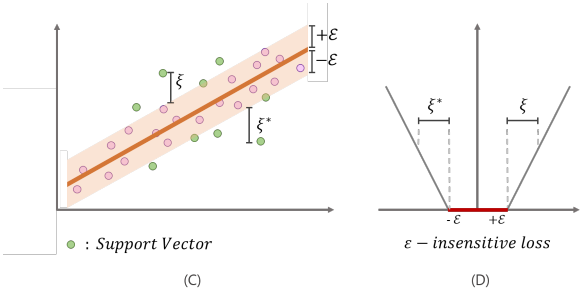
SVR – противоестественно, но почему бы и да?

1. SVM адаптируется под задачу регрессии следующим образом: отыскивается гиперплоскость, от которой все точки выборки лежат не дальше, чем на ε:

$$\begin{cases} \forall\, i \;\; y_i - \vec{\mathbf{w}}^\top \vec{\mathbf{x}}_i - b \le \varepsilon + \xi_i \\ \forall\, i \;\; y_i - \vec{\mathbf{w}}^\top \vec{\mathbf{x}}_i - b \ge -\varepsilon - \xi_i^* \end{cases}$$

2. Оптимизационная задача имеет вид:

$$\begin{cases} \frac{1}{2}\|\vec{\mathbf{w}}\|^2 + C\sum\limits_{i=1}^{n}(\xi_i + \xi_i^*) \to \min \\ \forall\, i \;\; y_i - \vec{\mathbf{w}}^\top \vec{\mathbf{x}}_i - b \le \varepsilon + \xi_i \\ \forall\, i \;\; y_i - \vec{\mathbf{w}}^\top \vec{\mathbf{x}}_i - b \ge -\varepsilon - \xi_i^* \\ \forall\, i \;\; \xi_i,\, \xi_i^* \ge 0 \end{cases} \iff \sum_{i=1}^{n}\Big\{0,\; |y_i - \vec{\mathbf{w}}^\top \vec{\mathbf{x}}_i - b| - \varepsilon\Big\}_+ + \frac{1}{2C}\|\vec{\mathbf{w}}\|^2 \to \min$$

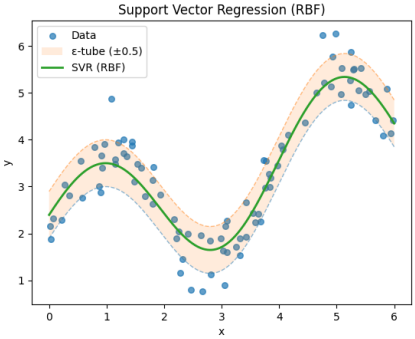


[source]

Kernelized SVR

Соответствующая двойственная задача (уже в ядерной постановке) имеет вид

$$-\frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n}(\lambda_i-\lambda_i^*)(\lambda_j-\lambda_j^*)\,K(\vec{\mathbf{x}}_i,\vec{\mathbf{x}}_j)-\varepsilon\sum_{i=1}^{n}(\lambda_i+\lambda_i^*)+\sum_{i=1}^{n}y_i(\lambda_i-\lambda_i^*)\to \max_{\substack{0\le\alpha_i,\alpha_i^*\le C\\ \sum(\alpha_i-\alpha_i^*)=0}}$$



[source]

О чем еще можно почитать

- Те, кому этот сюжет оказался интересен, могут почитать, чем отличаются реализации `SVC`, `LinearSVC` и `NuSVC` в `sklearn`, а также как можно быстро и эффективно делать kernel trick с помощью приближенного вычисления ядер
- Каким стандартным способом принято превращать расстояния до разделяющей поверхности в «вероятности» классов для SVC и какие тут есть подводные камни
- Какова связь между RBF-ядром, так называемыми Фурье-фичами и разложением функций в тригонометрические ряды
- Какая большая теория живет за ядрами и SVM'ом, а также как kernel trick можно реализовать для нелинейного снижения размерности (kernel PCA) и нелинейной регрессии (kernel Ridge Regression)

Спасибо за внимание

Буду рад ответить на ваши вопросы